

Module: `Problem 2 - Paying Debt off in a Year`

This module provides a single utility function that projects the remaining credit-card balance after a series of fixed minimum payments over a one-year period. It iteratively applies the monthly interest to the unpaid portion of the balance and updates the month counter until twelve months have been processed.

Functions

`calculate(month, balance, minPay, monthlyInterestRate)`

Description

Simulates the evolution of a credit-card balance over the remaining months of a year. Starting from the supplied `month` index, the function repeatedly:

1. Subtracts the fixed minimum payment `minPay` from the current `balance` to obtain the unpaid portion.
2. Applies the monthly interest (`monthlyInterestRate`) to that unpaid portion.
3. Updates the balance with the interest-augmented amount.
4. Increments the month counter.

The loop stops once `month` reaches 12, and the final balance is returned.

Parameters

- `month` (*int*): The current month index (0-based or 1-based). The loop continues while `month < 12`.
- `balance` (*float*): The outstanding balance at the start of the current month.
- `minPay` (*float*): The fixed minimum payment applied each month.
- `monthlyInterestRate` (*float*): The monthly interest rate expressed as a decimal (e.g., 0.015 for 1.5% per month).

Returns

- `balance` (*float*): The projected balance after processing up to month 12. If the loop runs the full twelve iterations, this is the balance after one year of payments.

Examples

```
# Example: starting in month 0 with a $5,000 balance,
# paying $150 each month, and a monthly interest rate of 1.5%
final_balance = calculate(
    month=0,
    balance=5000.0,
    minPay=150.0,
    monthlyInterestRate=0.015
)
print(f"Balance after one year: ${final_balance:,.2f}")
# → Balance after one year: $4,328.57 (illustrative output)
```

Note: The exact numeric result depends on the precise arithmetic and rounding behavior of Python's float operations.

Edge Cases / Notes

- **Month start value:** If `month` is already `>= 12`, the loop body is never executed and the original `balance` is returned unchanged.
- **Negative unpaid balance:** If `minPay` exceeds the current `balance`, `unpaidBalance` becomes negative, causing the interest calculation to increase the negative amount (i.e., the balance may become more negative). The function does not guard against over-payment.
- **Interest rate of zero:** With `monthlyInterestRate = 0.0`, the balance simply decreases by `minPay` each month.
- **Floating-point precision:** Because the function uses Python's binary floating-point arithmetic, very small rounding errors may accumulate over the twelve iterations. For financial calculations, consider using `decimal.Decimal`.